Parallel.Project.Structure

Tech Stack

Layer	Choice
Backend	Java + Spring Boot
Frontend	React
Database	CockroachDB
Web Services	Spring Boot REST

Architecture

Monolith + Distributed DB — one Spring Boot application, CockroachDB partitioned across nodes.

Partitioning strategy:

- Users and Items — horizontal partitioning by user ID range or region
- Transactions — partitioned by date range

Team Structure

Subteam	Size	Owns
Backend	4	Spring Boot, business logic, REST APIs
Frontend	2	UI, user flows, screens
Database	1	Schema, distributed design, partitioning
DevOps/Integration	2	GitHub, coordination, report, diagrams

Sub-team Roles & Objectives

Backend

- Design the API contract first (api-contract.md) before writing any code

- Implement all 9 core features (auth, items, purchase, search, reports, etc.)
- Implement 3 SOAP/REST web service features (requirement x)
- Handle atomic transaction integrity for purchases (money + item transfer)
- Integrate with the DB layer

Frontend

- Build all UI screens for user-facing features
- Start on layout and screens using mock data before backend is ready
- Consume backend REST APIs as they become available
- Own end-to-end user flow coherence

Database

- Design the full schema (Users, Items, Transactions, Inventory, etc.)
- Decide and implement the partitioning strategy
- Deliver `schema.sql`, `partitioning.sql`, and `seed.sql`
- Document justification for every partitioning decision

DevOps/Integration

- Manage GitHub — branching strategy, PRs, protect main and dev
- Own the report (.docx) and presentation (.pptx)
- Build and maintain the Gantt chart
- Wire frontend to backend as features complete and catch integration breakage early
- Own demo session preparation

GitHub Branching Strategy

```
main
└── dev
├── feature/auth
├── feature/items
├── feature/purchase
├── feature/search
├── feature/reports
├── feature/inventory
├── feature/account
├── feature/webservices
└── feature/frontend-[screen-name]
```

Rules:

- main — production-ready only, DevOps merges here at milestones
- dev — integration branch, all features merge here first
- feature/* — one branch per feature
- No direct pushes to main or dev — pull requests only
- PR to dev requires at least 1 reviewer approval

PR Flow:

feature/auth → (PR + review) → dev → (tested + stable) → main

---

Directory Structure

```
marketplace/
├── backend/
│   ├── src/
│   │   └── main/
│   │       ├── java/com/marketplace/
│   │       │   ├── auth/
│   │       │   │   ├── AuthController.java
│   │       │   │   ├── AuthService.java
│   │       │   │   └── AuthRepository.java
│   │       │   ├── items/
│   │       │   │   ├── ItemController.java
│   │       │   │   ├── ItemService.java
│   │       │   │   └── ItemRepository.java
│   │       │   ├── purchase/
│   │       │   ├── search/
│   │       │   ├── inventory/
│   │       │   ├── reports/
│   │       │   ├── account/
│   │       │   ├── webservices/
│   │       │   └── shared/
│   │       │   ├── models/
│   │       │   └── exceptions/
│   │       └── resources/
│   │           └── application.properties
│   └── pom.xml
│
```

```
├── frontend/
│  ├── src/
│  │  ├── pages/
│  │  ├── components/
│  │  ├── services/
│  │  └── assets/
│  └── package.json
│
├── database/
│  ├── schema.sql
│  ├── seed.sql
│  └── partitioning.sql
│
├── docs/
│  ├── report.docx
│  ├── presentation.pptx
│  ├── api-contract.md
│  └── diagrams/
│  ├── architecture.png
│  ├── user-flow.png
│  └── db-schema.png
│
├── README.txt
└── .gitignore
```

Development Model

Lightweight Agile — 3 sprints after Phase 1, with a Discord check-in every 2 days. DevOps/Integration subteam acts as de facto Scrum Master.

Sprint	Dates	Focus
1	Apr 25 – May 1	Bootstrap + Auth + Account + Items
2	May 2 – May 8	Search + Purchase + Deposit + Inventory
3	May 9 – May 16	Reports + Web Services + Integration + Testing

---

Action Plan

Phase 1 — Design (Now → Apr 25)

Step 1 — Team Setup

- [x] ~~Create GitHub repo, add all members~~
- [x] ~~Create Discord channels: #general, #backend, #frontend, #database, #devops~~
- [ ] Create Miro board
- [ ] Assign sub-teams and leads
- [ ] Share and read requirements document together

Step 2 — Architecture Decision

- [x] ~~Agree on final tech stack (Java Spring Boot, React/HTML, CockroachDB)~~
- [x] ~~Agree on architecture: monolith + distributed DB~~
- [ ] Draw system block diagram on Miro
- [ ] Draw user flow for each feature (i–ix) on Miro

Step 3 — DB Design

- [ ] List all entities (User, Item, Transaction, Inventory, etc.)
- [ ] Draw ER diagram
- [ ] Write initial schema.sql
- [ ] Decide partitioning strategy per table and document justification

Step 4 — Phase 1 Document

- [ ] Create report.docx with cover page and table of contents
- [ ] Write Introduction section
- [ ] Write System Architecture section (include block diagram)
- [ ] Write Distributed DB Design section (include ER + partitioning)
- [ ] Write initial Time Plan + Gantt chart
- [ ] Export all Miro diagrams as images into docs/diagrams/

Step 5 — Submit Phase 1

- [ ] Review document as a group
- [ ] Submit by Apr 25

Between Phase 1 and 2 — Build (Apr 25 → May 16)

Step 6 — Project Bootstrap

- [ ] Initialize Spring Boot project, push to dev
- [ ] Initialize frontend project, push to dev
- [ ] Set up CockroachDB locally for all backend developers
- [ ] Finalize and share api-contract.md before coding starts

Step 7 — DB Team

- [ ] Finalize schema.sql based on Phase 1 feedback
- [ ] Write partitioning.sql
- [ ] Write seed.sql with sample data for testing
- [ ] Set up a shared DB instance everyone can connect to

Step 8 — Backend (in this order)

- [ ] Auth — register, login, JWT or session
- [ ] Account — view balance, purchased/sold items
- [ ] Items — add, edit, remove, list
- [ ] Deposit — add cash to account
- [ ] Search — text search by name and brand
- [ ] Purchase — atomic money + item transfer
- [ ] Inventory — manage stock
- [ ] Reports — transaction reports
- [ ] Web Services — pick 3 features, expose as REST endpoints

Step 9 — Frontend (parallel with backend)

- [ ] Build all screens using mock data first
- [ ] Screens: Register, Login, Dashboard, Item Listing, Search, Purchase, Deposit, Inventory, Reports
- [ ] Replace mock data with real API calls as backend features complete

Step 10 — Integration

- [ ] Wire frontend to backend feature by feature as each completes
- [ ] Fix any contract mismatches between frontend and backend
- [ ] Test full user flows end to end

Step 11 — Testing

- [ ] Write component tests for each backend service
- [ ] Test all API endpoints (use Postman)
- [ ] Test all frontend screens
- [ ] Document all test cases and results for the report

Phase 2 — Documentation & Demo (→ May 16)

Step 12 — Final Report

- [ ] Update all sections from Phase 1 with final decisions
- [ ] Write Application Level Protocol section
- [ ] Write Testing section with all test cases and results
- [ ] Write End-User Guide with screenshots
- [ ] Write Role of Each Member section
- [ ] Write Resources section
- [ ] Add References

Step 13 — Presentation

- [ ] Build presentation.pptx covering all project phases
- [ ] Prepare live demo script
- [ ] Rehearse demo as a group

Step 14 — Final Submission

- [ ] Zip: source code, schema.sql, executables, resource files
- [ ] Include README.txt with compile and run instructions
- [ ] Submit on LMS by May 16